

CSC10 Eindopdracht

Reactiespel



Krzysztof Cajler (0951562)

Thomas van Egmond (1038854)

Vak: CSC10

Datum: 25.01.2026

Versie: 1.0

Samenvatting

In dit verslag beschrijven we de realisatie van een embedded systeem op een het DE1-SoC bord. Het doel was om een reactiespel te maken en dit op twee manieren aan te sturen: via een Soft Core (Nios II) en via een Hard Core (ARM met Linux). Uit de opdracht blijkt dat beide systemen het spel prima kunnen draaien. Echter, voor een applicatie die zo dicht op de hardware zit, is de Nios II de meest logische keuze omdat deze veel minder overhead heeft dan Linux.

Inhoud

Samenvatting.....	2
Inleiding	4
Deelopdracht 1	4
Deelopdracht 2	4
Reactie spel component.....	5
De registers	5
Deelopdracht 1: Reactiespel met behulp van de softcore en RTOS.....	6
De software.....	7
Task 1: Game task.....	7
Task 2: Config task	7
Task 3: Display task.....	7
Main	7
Deelopdracht 2: Reactiespel met behulp van Linux en Kernel modules	8
Kernel Driver	10
C-applicatie	10
Vergelijking	11
Conclusie.....	11
Bibliografie	12

Inleiding

Voor de eindopdracht van het vak CSC10 zijn er twee deelopdrachten bedacht om alle vijf leerdoelen te kunnen bewijzen. Het resultaat is vrijwel dezelfde: een reactiespelletje, met behulp van ledjes, switches en buttons die op de FPGA beschikbaar zijn. Echter zijn de registers en de werking van het spel net iets geworden, dan bij het voorstel van de eindopdracht. Dit heeft als reden dat de toepassing van het voorstel eigenlijk geen verschil zou maken in de snelheid van de software, dus geen verschil tussen nios II en kernel module. In het voorstel was het beschreven dat gehele spellogica in de hardware zich bevond. Uiteindelijk hebben we besloten om de spellogica te splitsen: de controle van LED's gaat in de hardware gebeuren, de hardware bepaalt aan de hand van de staat van het register hoe het spel zich moet gaan gedragen. Echter het lezen van de knop gebeurt door middel van een interrupt in de software. In de software wordt dus vergeleken of de LED op de juiste plek zich bevond. Het spel gaat als volgt:

1. De switches bepalen de instelling van het spel. SW (0-7) bepaalt de snelheid van LEDs. SW (8) maakt het mogelijk om een modus te kiezen: heen en weer of door laten gaan (van led 9 naar 0). Uiteindelijk zorgt SW (9) voor het starten en stoppen van het spel
2. Wanneer spel gaande is, wordt KEY (3) gebruikt om te kunnen spelen, target wordt ingesteld in software op positie 4 en 5. Wanneer het knopje gedrukt wordt en het lampje staat op de target, wordt de score verhoogt en weergegeven op de HEX display. De spel kan met KEY (0) gereset worden.

Deelopdracht 1

Deelopdracht 1 maakt gebruik van een eigen hardwarecomponent. Daarnaast wordt er ook een gemaakte module gebruikt die zorgt voor het schrijven naar de HEX display. Dat is de module die in de weekopdracht is gemaakt [1]. Daarnaast zijn er standaard componenten die nodig zijn om volledige systeem te kunnen bouwen zoals: nios II/e softcore, on-chip memory, JTAG UART en Parallel I/O's. Deze softcore maakt ook gebruik RTOS microC. Deze opdracht zal dus leerdoel 1, 2 en 3 bewijzen. Samen met deelopdracht 2, wordt ook leerdoel 5 bewezen.

Deelopdracht 2

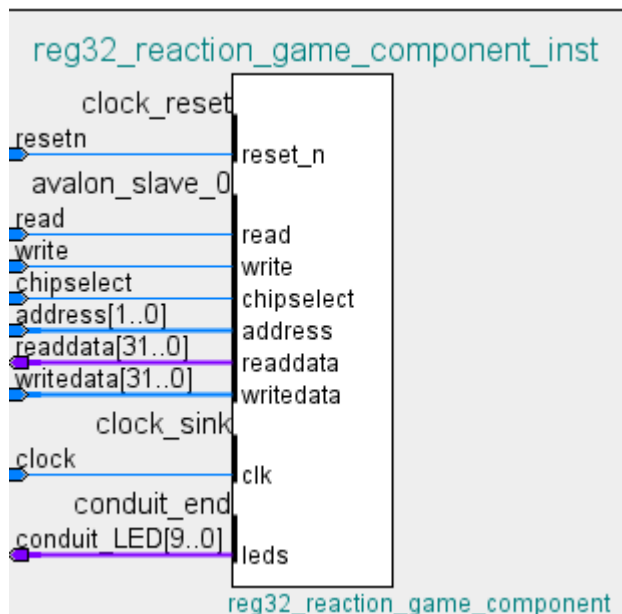
Deze deelopdracht maakt ook gebruik van dezelfde zelfgemaakte componenten. Deze opdracht wordt op Linux uitgevoerd en wordt aangemaakt als een kernel module. Deze opdracht bewijst leerdoel 1, 4 en samen met deelopdracht leerdoel 5.

Reactie spel component

Het reactiespel component is een Avalon Memory-Mapped Interface, deze is in Figuur 1 te zien. Het heeft benodigde clock, reset en signalen zoals read, write, chipselect, address, readdata en writedata. Deze address is van belang, omdat deze wordt gebruikt om een bepaalde registers van spellogica te kiezen. De output van deze component is LED-array, te zien als conduit_LED[9..0].

De registers

De controle van het spel gaat door middel van de registers. Er zijn zoals in het voorstel aangegeven drie registers in de component aanwezig, namelijk: CTRL, speed en status. In Tabel 1 is het overzicht van deze registers te zien.



Figuur 1 Signalen van eigen component

Offset	Register Naam	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x00	CTRL	RESERVED															TARGET[9:0]										RESERVED						MODE		ENABLE
0x04	SPEED	SPEED[31:0]																																	
0x08	STATUS	RESERVED																						CURRENT POSITION[9:0]											

Tabel 1 De registers van eigen component

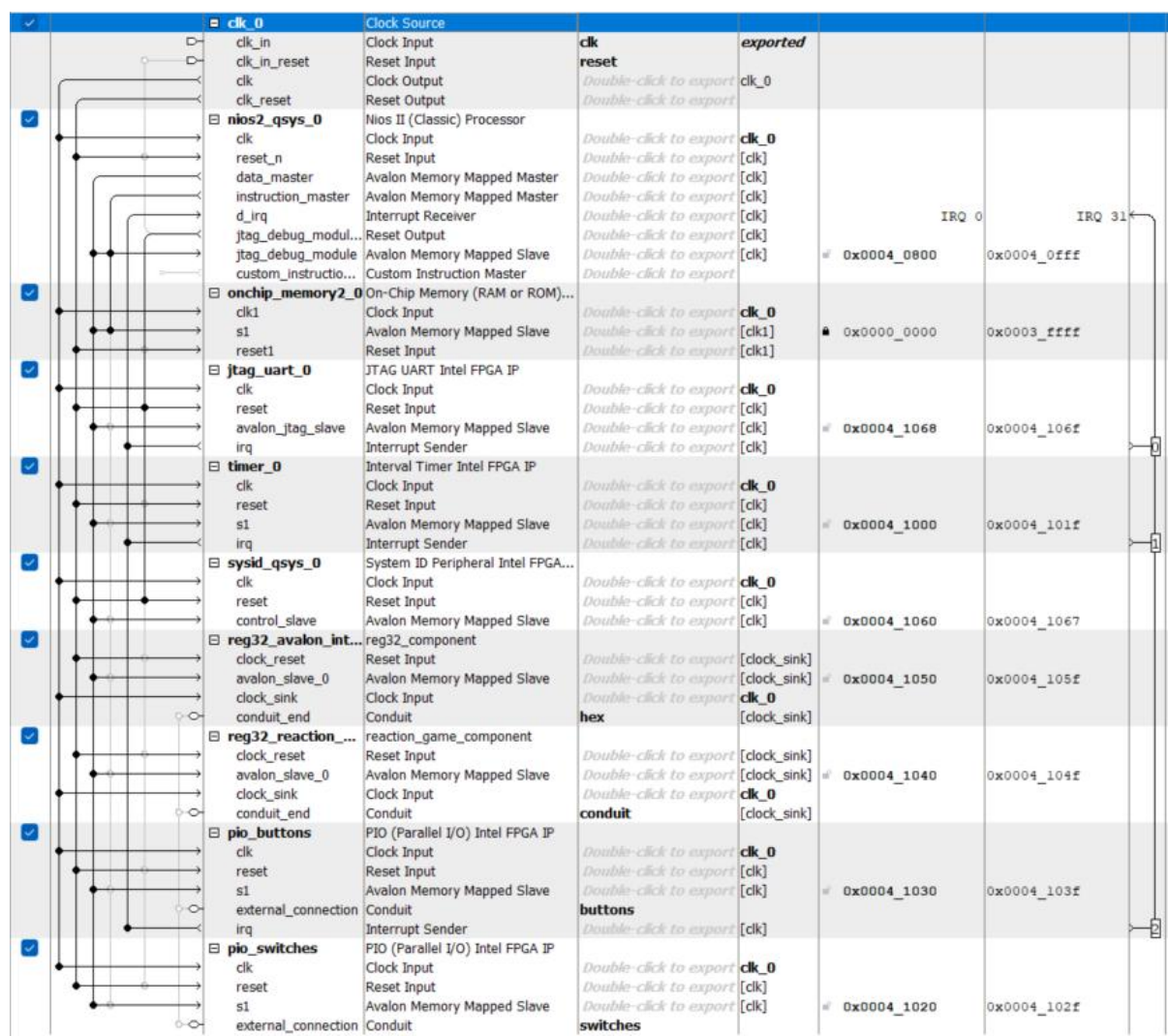
Zoals in tabel te zien is, zouden de bits van huidige positie in CTRL-register passen. Echter is dat niet gedaan, omdat het aanpassen van huidige positie in software niet gewenst is. Daarom is het een aparte readonly register.

De hardware bevat paar processen om de logica van component te beschrijven, dat zijn:

- Write process, hiermee wordt de data naar de registers geschreven
- Read process, hiermee wordt afhankelijk van gekozen address de register gelezen
- LED process, hiermee wordt de huidige positie naar conduit_LED geschreven
- Game process, hiermee gebeurt de hele logica van de spel, hoe de leds zich verplaatsen

Deelopdracht 1: Reactiespel met behulp van de softcore en RTOS

Voordat de reactiespel geprogrammeerd kan zijn, moet het systeem in platform designer ontworpen zijn. Deze is in Figuur 2 te zien. Het heeft standaard componenten als de klok, nios II softcore, on chip memory, uart, (optionele) system id en twee avalon memory mapped interfaces. Een ervan is bedoeld voor de hex display (weekopdracht 2) en de andere is de reactiespel. Daarnaast zijn Parallel I/O's gedefinieerd voor button en switches.



Figuur 2 De componenten van softcore implementatie

De software

Het reactiespel wordt op nios II met behulp van RTOS microC, die standaard beschikbaar is. Ook wordt er gebruikgemaakt BSP en HAL. De software beschikt over drie taken met verschillende prioriteiten

Task 1: Game task

Deze task is de spellogica. Deze task wacht op de interrupt van de button en daarna leest deze zowel status- als de ctrl-register. De task wacht dus op de interrupt die de semaphore post, daarmee weet deze taak dat die verder uitgevoerd kan worden. Door bitmasking wordt de huidige positie en de target uitgehaald. Wanneer deze overeenkomen, wordt de globale score verhoogd. Doordat deze de daadwerkelijke reactie van de speler test, moet deze task de hoogste prioriteit krijgen.

Task 2: Config task

Deze task is constant de positie van de switches aan het lezen. Hier kan het spel gestart/gestopt worden, modus aangepast en de snelheid. Het lezen gebeurt een keer per 100 ms. Als er een aanpassing zich heeft bevonden, wordt door bit manipulatie alleen veranderde setting aangepast.

Task 3: Display task

Deze task stuurt de HEX-display aan door middel van de avalon memory mapped interface die in de weekopdracht 2 is gemaakt [1]. Deze taak krijgt de laagste prioriteit, omdat het weergeven van de score altijd iets later kan gebeuren.

Main

In de main worden alle taken geïnitialiseerd, twee semaphoren gecreëerd en de interrupt geregistreerd. Als dit gedaan is, wordt de OS gestart.

Deelopdracht 2: Reactiespel met behulp van Linux en Kernel modules

Voor de Linux-versie hebben we dezelfde VHDL-hardware gebruikt. In Platform Designer hebben we de Nios II weggehaald en alles gekoppeld aan de HPS Lightweight AXI Bridge. Zoals we hebben geleerd in de opdrachten van week 3, om de FPGA te verbinden met de Hard Processor System [2]. Hierdoor worden de adressen van onze componenten (LEDs, PIO's) beschikbaar in de memory map van de ARM-processor (afbeelding op volgende pagina).

Om ervoor te zorgen dat Linux de hardware ziet hebben we de Device Tree Blob aangepast. We hebben het genereren van een .dts bestand en het toevoegen van compatible strings gedaan zoals in week 3 [2]. En deze vervolgens omgezet naar een .dtb.

```
moving_led_0: moving_led_0@0x100000010 {
    compatible = "csc10,reactiespel";
    reg = <0x00000001 0x00000010 0x00000010>;
    clocks = <&clk_0>;
};

a_7_segment_encoder_0: a_7_segment_encoder_0@0x100000020 {
    compatible = "csc10,7_segment";
    reg = <0x00000001 0x00000020 0x00000010>;
    clocks = <&clk_0>;
};

buttons_pio: gpio@0x100000000 {
    compatible = "csc10,button";
    reg = <0x00000001 0x00000000 0x00000010>;
    interrupt-parent = <&hps_0_arm_gic_0>;
    interrupts = <0 40 1>;
    clocks = <&clk_0>;

etc.
```


Platform Designer - hps_systeem.qsys (C:\Users\tvane\Desktop\School\Jaar4\CSC10\hps_demo\hps_systeem.qsys)

File Edit System Generate View Tools Help

IP Catalog

Project

New Component...

Library

- 7_segment_encoder
- moving_led
- Basic Functions
- DSP
- Interface Protocols
- Low Power
- Memory Interfaces and Controllers
- Processors and Peripherals
- Qsys Interconnect
- System
- Tri-State Components
- University Program

New... Edit... Add...

System Contents

Address Map

Interconnect Requirements

System: hps_systeem Path: clk_0

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
☒		clk_0	Clock Source		exported					
☒		clk_in	Clock Input	clk						
☒		clk_in_reset	Reset Input	reset						
☒		clk	Clock Output		clk_0					
☒		clk_reset	Reset Output							
☒		hps_0	Arria V/Cyclone V Hard Process...							
☒		memory	Conduit	memory						
☒		hps_io	Conduit	hps_io						
☒		h2f_reset	Reset Output							
☒		h2f_lw_axi_clock	Clock Input		clk_0					
☒		h2f_lw_axi_master	AXI Master		[h2f_lw_axi_clock]					
☒		f2h_irq0	Interrupt Receiver					IRQ 0	IRQ 31	
☒		f2h_irq1	Interrupt Receiver					IRQ 0	IRQ 31	
☒		moving_led_0	moving_led							
☒		avalon_slave_0	Avalon Memory Mapped Slave		[clock_sink]	0x0000_0010	0x0000_001f			
☒		reset_sink	Reset Input		[clock_sink]					
☒		conduit_end	Conduit		[clock_sink]					
☒		clock_sink	Clock Input		clk_0					
☒		buttons_pio	PIO (Parallel I/O) Intel FPGA IP							
☒		clk	Clock Input		clk_0					
☒		reset	Reset Input		[clk]					
☒		s1	Avalon Memory Mapped Slave		[clk]	0x0000_0000	0x0000_000f			
☒		external_connection	Conduit							
☒		irq	Interrupt Sender		[clk]					
☒		a_7_segment_en...	7_segment_encoder							
☒		avalon_slave_0	Avalon Memory Mapped Slave		[clock_sink]	0x0000_0020	0x0000_002f			
☒		conduit_end	Conduit							
☒		clock_sink	Clock Input		clk_0					
☒		reset_sink	Reset Input		[clock_sink]					
☒		switches_pio	PIO (Parallel I/O) Intel FPGA IP							
☒		clk	Clock Input		clk_0					
☒		reset	Reset Input		[clk]					
☒		s1	Avalon Memory Mapped Slave		[clk]	0x0000_0030	0x0000_003f			
☒		external_connection	Conduit							

Current filter:

Messages

Type	Path	Message
Warning	3 Warnings	
Warning	hps_systeem.hps_0	"Configuration/HPS-to-FPGA user 0 clock frequency" (desired_cfg_clk_mhz) requested 100.0 MHz, but only achieved 97.368421 MHz

0 Errors, 3 Warnings

Generate HDL... Finish

Figuur 3 De componenten van hardcore implementatie

Kernel Driver

Omdat Linux op een Hard Core draait, kunnen we niet direct bij de interrupts zoals op de Nios. Hiervoor hebben we een kernel module (button_driver) nodig, gebruikmakend van de kennis over kernel modules en interrupts uit weekopdracht 3 [2].

- De driver registreert de interrupt (IRQ 72).
- Bij een druk op de knop wordt de handler in de kernel aangeroepen.
- De driver stuurt vervolgens een SIGIO signaal naar ons programma in user space. Dit idee, fasync, hebben we overgenomen uit de opdrachten van week 3 [2].

C-applicatie

De applicatie in Linux is geschreven in C en gebruikt mmap om via /dev/mem direct naar de fysieke adressen van de FPGA te schrijven. We gebruiken hiervoor de eerdere code uit de week 3 opdrachten [2]. De applicatie probeert zo veel mogelijk één op één de Nios versie na te bootsen en werkt daarom ook in hele gelijke wijze. We hebben threads gebruikt om de taken van de Nios na te maken.

Tijdens het testen liepen we tegen een probleem aan. De driver zet bij het laden (via insmod) de interrupts aan in de hardware (door 0xF naar het Interrupt Mask Register van de PIO te schrijven). Echter als we op het bord op de reset knop (KEY0) drukten, werd de hardware gereset. Hierdoor ging het masker van de PIO terug naar 0 (uit). De driver weet dit niet en denkt dat alles nog aan staat. Waardoor de knoppen niet meer werkte na een reset.

We hebben de C-code in user space aangepast. Bij het opstarten van het spel schrijft de applicatie via mmap de waarde 0xF naar offset 0x8 (het mask register) van de button-PIO. Hierdoor worden de interrupts in de hardware weer aangezet, ook al was de driver al geladen. Waardoor de game ook na een reset nog werkt.

Vergelijking

Om te bepalen welke functionaliteit je het beste waar kunt implementeren hebben we gekeken naar: in hardware, op een soft core of op een hard core.

Volledig in Hardware (VHDL):

We hadden de volledige logica, dus ook scoretelling en knop-controle, in VHDL kunnen bouwen. Dat zou technisch perfect werken: het systeem zou snel reageren en stabiel zijn. Echter voor ingewikkeldere logica, zoals spelregels, is schrijven in VHDL is tijdrovend en moeilijker aan te passen.

Soft Core (Nios II):

De logica schrijven in C op een Soft Core gaat aanzienlijk sneller en is veel makkelijker aan te passen dan VHDL. Omdat de Nios II direct toegang heeft tot de hardware (zonder OS ertussen), is de reactiesnelheid nog steeds erg goed. Daarom is onze huidige oplossing ideaal: de tijd gebonden onderdelen zoals LED beweging in hardware en de logica in C op de Soft Core.

Hard Core (Linux):

De Linux oplossing werkt functioneel ook prima, maar is overkill. Het vereist een volledig besturingssysteem, device trees en complexe drivers om een knopje uit te lezen. Er zijn veel meer stappen (overhead) nodig om het signaal bij je C-code te krijgen. Een Hard Core met Linux is nuttig als je functies nodig hebt zoals netwerken, USB ondersteuning en dergelijke.

Conclusie

We hebben aangetoond dat het reactiespel op beide platforms goed werkt. De combinatie van Hardware voor de tijdgebonden onderdelen zoals LED beweging, en Software voor de spellogica is een goede opstelling.

Voor keuze tussen Soft Core Nios en Hard Core Linux gaat de voorkeur uit naar de Nios II. Voor low-level aansturing en simpele logica biedt de Soft Core het beste van beide werelden. Het ontwikkelt veel sneller dan VHDL en heeft veel minder overhead en complexiteit dan een Linux systeem.

Bibliografie

- [1] H. Rotterdam, „Opdrachten week 2 – Versie 1.7,” [Online]. Available:
https://bytebucket.org/HR_ELEKTRO/csc10/wiki/Opdrachten/Opdrachten_Week_2.pdf.
- [2] H. Rotterdam, „Opdrachten week 3 – Versie 1.7,” [Online]. Available:
https://bytebucket.org/HR_ELEKTRO/csc10/wiki/Opdrachten/Opdrachten_Week_3.pdf.